

Лабораторный практикум 15. Фролов А.А.

Переменные управления курсором режима редактирования

```
1 ; -----
2 ; Переменные режима редактирования сектора
3 ; B - номер редактируемого байта в Sector
4 ; Nt - номер редактируемой тетрады в байте:
5 ;      0 - первая тетрада, 1 - вторая тетрада
6 ; -----
7 B db 0
8 Nt db 0
```

Процедура Set_cursor вычисляет координаты курсора по номеру байта B и номеру тетрады Nt. Для установки курсора используется BIOS-прерывание int 10h с функцией 02h.

```
1 ; -----
2 ; Установка курсора на текущую тетраду байта Sector
3 ;
4 ; Использует:
5 ; B - номер байта в Sector
6 ; Nt - номер тетрады в байте: 0 или 1
7 ;
8 ; Результат:
9 ; курсор устанавливается в позицию дампа,
10 ; соответствующую байту B и тетраде Nt
11 ; -----
12 Set_cursor:
13     push ax
14     push bx
15     push dx
16
17     ; DH = номер строки дампа = B / 16
18     mov al,[B]
19     mov ah,0
20     mov dh,al
21     shr dh,4
22
23     ; DL = номер байта в строке = B mod 16
24     mov dl,al
25     and dl,0Fh
26
27     ; DL = DL * 3
28     ; так как каждый байт выводится как "XX "
29     mov bl,dl
30     shl dl,1
```

```

31      add dl,bl
32
33      ; DL = DL + Nt
34      add dl,[Nt]
35
36      ; Сдвиг дампа на экране:
37      ; 7 символов вправо и 8 строк вниз
38      add dl,7
39      add dh,8
40
41      ; BIOS: установка позиции курсора
42      mov ah,02h
43      mov bh,0
44      int 10h
45
46      pop dx
47      pop bx
48      pop ax
49      ret

```

Cur_right – если курсор был на первой тетраде, переходит на вторую. Если был на второй, переходит на первый полубайт следующего байта.

Cur_left – если курсор был на второй тетраде, переходит на первую. Если был на первой, переходит на второй полубайт предыдущего байта.

Cur_top – уменьшает номер байта на 16, то есть поднимает курсор на строку выше.

Cur_down – увеличивает номер байта на 16, то есть опускает курсор на строку ниже.

```

1      ; -----
2      ; Перемещение курсора на одну тетраду вправо
3      ; -----
4      Cur_right:
5          cmp byte ptr [Nt],1
6          je Cur_right_next_byte
7
8          inc byte ptr [Nt]
9          jmp Cur_right_exit
10
11     Cur_right_next_byte:
12         mov byte ptr [Nt],0
13         inc byte ptr [B]
14
15     Cur_right_exit:
16         call Set_cursor
17         ret
18
19
20     ; -----

```

```

21 ; Перемещение курсора на одну тетраду влево
22 ; -----
23 Cur_left:
24     cmp byte ptr [Nt],0
25     je Cur_left_prev_byte
26
27     dec byte ptr [Nt]
28     jmp Cur_left_exit
29
30 Cur_left_prev_byte:
31     mov byte ptr [Nt],1
32     dec byte ptr [B]
33
34 Cur_left_exit:
35     call Set_cursor
36     ret
37
38
39 ; -----
40 ; Перемещение курсора на одну строку вверх
41 ; -----
42 Cur_top:
43     sub byte ptr [B],16
44     call Set_cursor
45     ret
46
47
48 ; -----
49 ; Перемещение курсора на одну строку вниз
50 ; -----
51 Cur_down:
52     add byte ptr [B],16
53     call Set_cursor
54     ret

```

Процедура Edit_command используется внутри режима редактирования сектора. Она получает скан-код управляющей клавиши, ищет его в таблице Edit_table и вызывает одну из процедур перемещения курсора: Cur_right, Cur_left, Cur_top или Cur_down.

```

1 ; -----
2 ; Интерпретатор команд режима редактирования
3 ;
4 ; Вход:
5 ; AH = скан-код нажатой управляющей клавиши
6 ; -----
7 Edit_command:
8     push bx
9     push dx

```

```

10
11     mov dl,ah
12     mov bx,offset Edit_table
13
14 Edit_command_loop:
15     cmp byte ptr [bx],0FFh
16     je Edit_command_exit
17
18     cmp dl,[bx]
19     je Edit_command_dispatch
20
21     add bx,3
22     jmp Edit_command_loop
23
24 Edit_command_dispatch:
25     inc bx
26     call word ptr [bx]
27
28 Edit_command_exit:
29     pop dx
30     pop bx
31     ret
32
33
34 ; -----
35 ; Таблица команд режима редактирования
36 ; -----
37 Edit_table db 4Dh                ; стрелка вправо
38            dw Cur_right
39
40            db 4Bh                ; стрелка влево
41            dw Cur_left
42
43            db 48h                ; стрелка вверх
44            dw Cur_top
45
46            db 50h                ; стрелка вниз
47            dw Cur_down
48
49            db 0FFh

```

Процедура Check_digit_hex получает ASCII-код символа в регистре AL и проверяет, является ли он шестнадцатеричной цифрой. Если символ корректный, его числовое значение возвращается в регистре DL.

```

1  ; -----
2  ; Проверка шестнадцатеричной цифры
3  ;
4  ; Вход:

```

```

5 ; AL = ASCII-код символа
6 ;
7 ; Выход:
8 ; CF = 0, если символ является hex-цифрой
9 ; DL = значение цифры от 0 до 15
10 ;
11 ; CF = 1, если символ не является hex-цифрой
12 ; -----
13 Check_digit_hex:
14     cmp al,'0'
15     jnb Check_digit_hex_bad
16
17     cmp al,'9'
18     jbe Check_digit_hex_09
19
20     cmp al,'A'
21     jnb Check_digit_hex_small
22
23     cmp al,'F'
24     jbe Check_digit_hex_AF
25
26 Check_digit_hex_small:
27     cmp al,'a'
28     jnb Check_digit_hex_bad
29
30     cmp al,'f'
31     jbe Check_digit_hex_af
32
33     jmp Check_digit_hex_bad
34
35
36 Check_digit_hex_09:
37     sub al,'0'
38     mov dl,al
39     cld
40     ret
41
42
43 Check_digit_hex_AF:
44     sub al,'A'
45     add al,10
46     mov dl,al
47     cld
48     ret
49
50
51 Check_digit_hex_af:
52     sub al,'a'
53     add al,10
54     mov dl,al

```

```

55     clc
56     ret
57
58
59 Check_digit_hex_bad:
60     stc
61     ret

```

Эта процедура будет записывать введенную hex-цифру в текущую тетраду байта Sector[B]

```

1  ; -----
2  ; Запись шестнадцатеричной цифры в Sector
3  ;
4  ; Вход:
5  ; DL = значение цифры от 0 до 15
6  ;
7  ; Использует:
8  ; B  - номер байта в Sector
9  ; Nt - номер тетрады в байте
10 ; -----
11 Write_digit_sector:
12     push ax
13     push bx
14     push dx
15
16     xor bx,bx
17     mov bl,[B]           ; BX = номер редактируемого байта
18
19     mov al,[Sector+bx]   ; AL = текущий байт Sector[B]
20
21     cmp byte ptr [Nt],0
22     jne Write_low_tetrad
23
24 Write_high_tetrad:
25     ; Редактируем первую тетраду байта
26     ; Сохраняем младшую тетраду, старшую очищаем
27     and al,0Fh
28
29     ; DL содержит цифру 0..15
30     ; переносим её в старшую тетраду
31     mov ah,dl
32     shl ah,4
33
34     or al,ah
35     jmp Write_digit_save
36
37 Write_low_tetrad:
38     ; Редактируем вторую тетраду байта

```

```

39      ; Сохраняем старшую тетраду, младшую очищаем
40      and al,0F0h
41
42      ; Записываем цифру в младшую тетраду
43      or al,dI
44
45  Write_digit_save:
46      mov [Sector+bx],al
47
48      ; После записи переходим к следующей тетраде
49      call Cur_right
50
51      ; После изменения заново выводим Sector
52      call disp_sector
53
54      ; И снова ставим курсор на текущую позицию
55      call Set_cursor
56
57      pop dx
58      pop bx
59      pop ax
60      ret

```

Процедура Edit_sector реализует основной цикл режима редактирования сектора. Она инициализирует переменные B и Nt, выводит Sector на экран, устанавливает курсор и ожидает нажатия клавиш. Клавиши Esc и F4 завершают редактирование, стрелки передаются процедуре Edit_command для перемещения курсора, а шестнадцатеричные цифры после проверки записываются в текущую тетраду байта с помощью процедуры Write_digit_sector.

```

1      ; -----
2      ; Редактирование сектора
3      ;
4      ; F4 или Esc - выход из режима редактирования
5      ; Стрелки - перемещение курсора
6      ; 0-9,A-F - запись hex-цифры в текущую тетраду
7      ; -----
8  Edit_sector:
9      ; инициализация позиции редактирования
10     mov byte ptr [B],0
11     mov byte ptr [Nt],0
12
13     call disp_sector
14     call Set_cursor
15
16  Edit_sector_loop:
17     ; ожидание нажатия клавиши
18     mov ah,00h

```

```

19      int 16h                ; AL = ASCII-код, AH = скан-код
20
21      ; Esc?
22      cmp ax,011Bh
23      je Edit_sector_exit
24
25      ; F4?
26      cmp ax,3E00h
27      je Edit_sector_exit
28
29      ; управляющая клавиша?
30      cmp al,00h
31      jne Edit_sector_check_hex
32
33      ; если управляющая клавиша, передаём её в Edit_command
34      call Edit_command
35      jmp Edit_sector_loop
36
37 Edit_sector_check_hex:
38      ; проверяем, является ли символ hex-цифрой
39      call Check_digit_hex
40      jc Edit_sector_loop      ; если не hex-цифра, игнорируем
41
42      ; если hex-цифра, DL = значение 0..15
43      call Write_digit_sector
44      jmp Edit_sector_loop
45
46 Edit_sector_exit:
47      ret

```


Компиляция:

```
DOSBox-X 2026.01.02: DN - 3000 cycles/ms
Main CPU Video Sound DOS Drive Capture Debug Help
14:36:28

Assembling and linking COM file...

Turbo Assembler Version 4.1 Copyright (c) 1988, 1996 Borland International

Assembling file: disp_sec.ASM
Error messages: None
Warning messages: None
Passes: 1
Remaining memory: 428k

Turbo Link Version 7.1.30.1. Copyright (c) 1987, 1996 Borland International

C:\PRAC15>
F1 Help F10 Menu
```

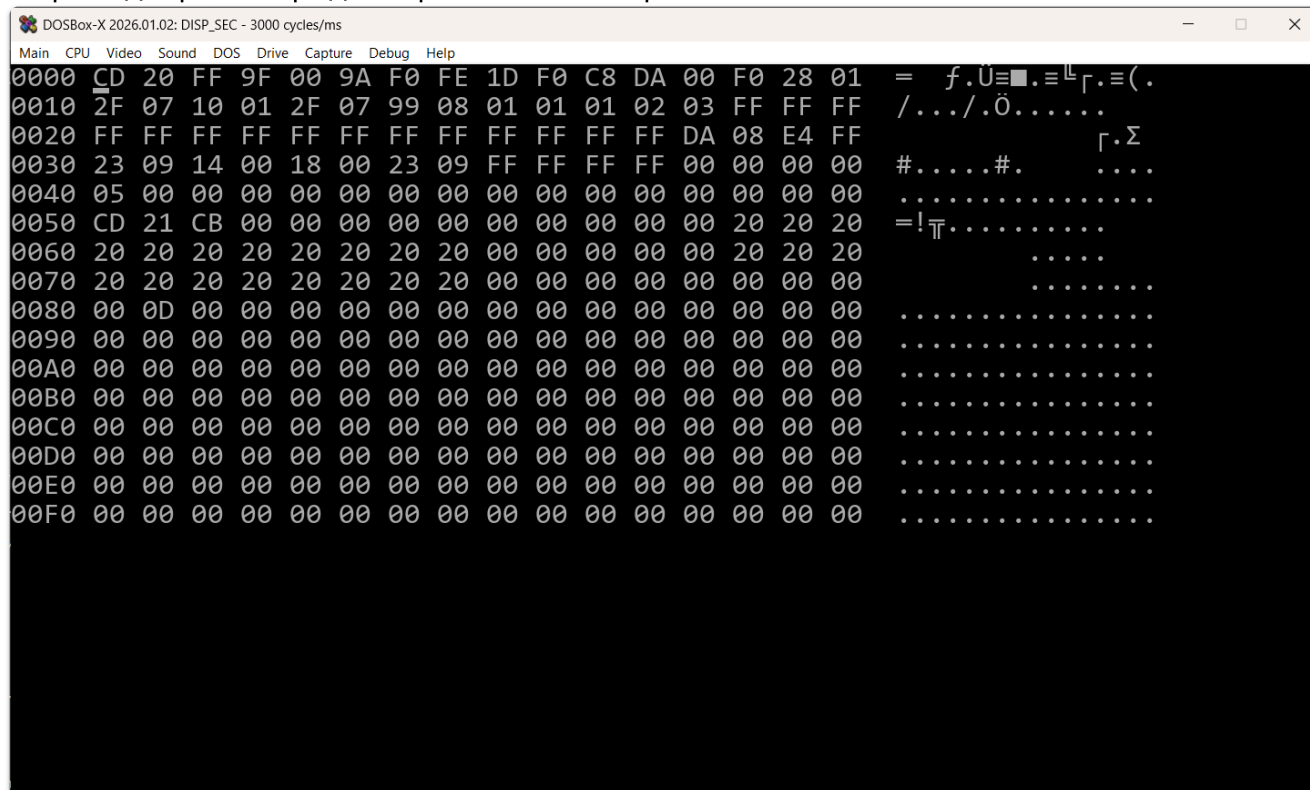
Начальный сектор (F1)

```
DOSBox-X 2026.01.02: DISP_SEC - 3000 cycles/ms
Main CPU Video Sound DOS Drive Capture Debug Help

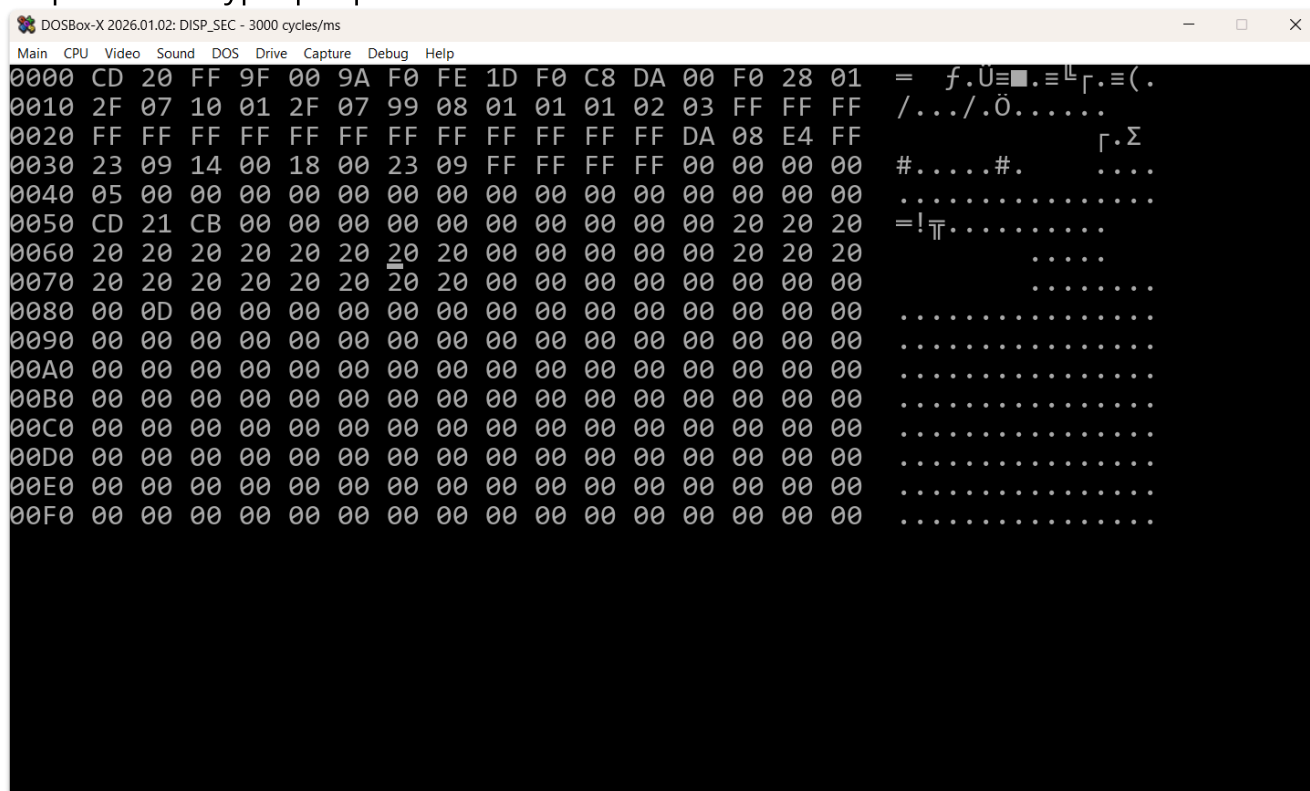
0000 CD 20 FF 9F 00 9A F0 FE 1D F0 C8 DA 00 F0 28 01 = f.Ü■.≡ℒΓ.≡(.
0010 2F 07 10 01 2F 07 99 08 01 01 01 02 03 FF FF FF /.../.Ö.....
0020 FF FF FF FF FF FF FF FF FF FF FF DA 08 E4 FF Γ.Σ
0030 23 09 14 00 18 00 23 09 FF FF FF FF 00 00 00 00 #.....#. ....
0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0050 CD 21 CB 00 00 00 00 00 00 00 00 00 00 20 20 20 =!⌘.....
0060 20 20 20 20 20 20 20 20 00 00 00 00 00 20 20 20 .....
0070 20 20 20 20 20 20 20 20 00 00 00 00 00 00 00 00 .....
0080 00 0D 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0090 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00B0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00D0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00F0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

>
```

Переход в режим редактирования сектора по клавише F4



Переместил курсор стрелками



[illegible][illegible]